

Final Activity 3: SumoBot Challenge

Members:

Inocencio, Ron Alexander

Martinez, Nathan John

Recto, Jason Jay

Ventura, Carl Vincent

Subject: Robotics

Section: BSCS 3A

OBJECTIVE:

Design, build, and program an autonomous SumoBot that can detect opponents, stay inside the arena, and push the opposing robot out of the ring.

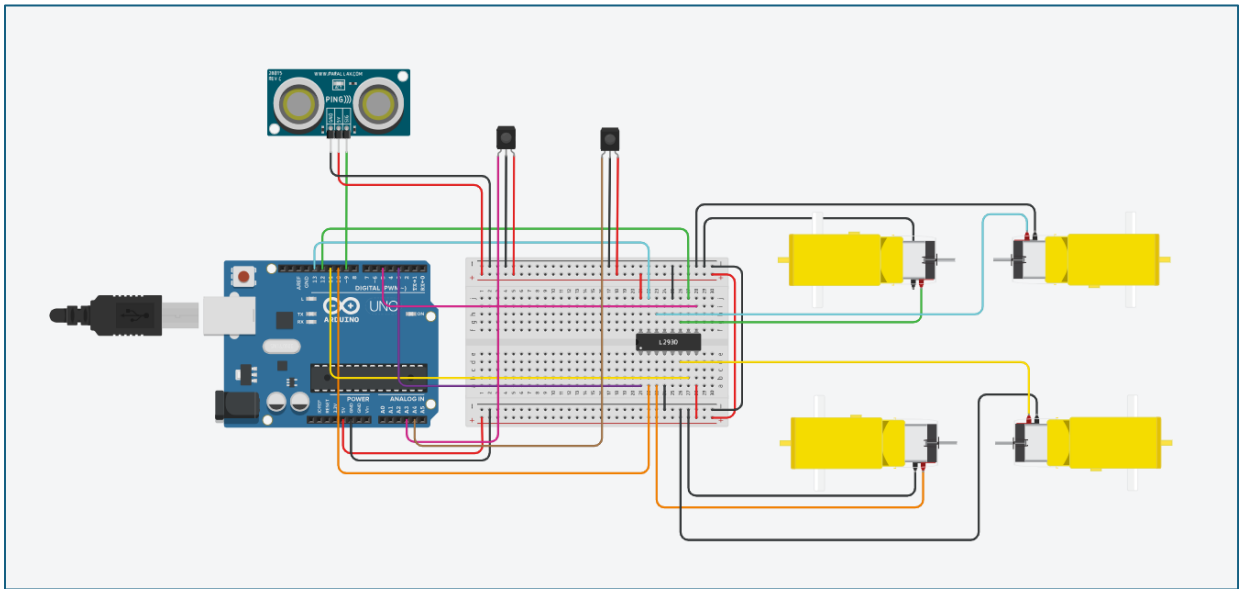
CONCEPT OVERVIEW:

A SumoBot is an autonomous robot inspired by sumo wrestling. Its goal is to locate an opponent inside a circular arena and push the opponent outside the boundary while avoiding falling off the edge itself.

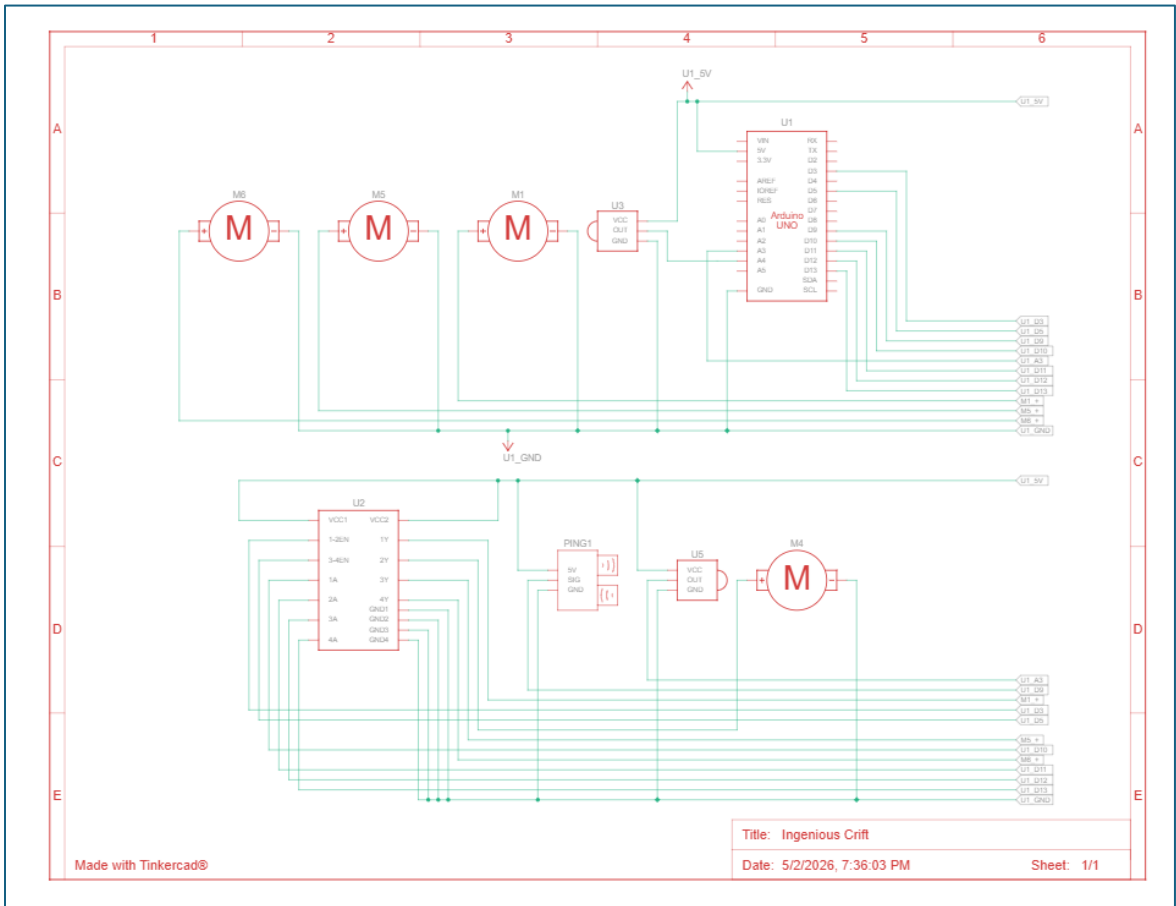
Components Used:

- Arduino Uno R3
- L293D Motor Driver IC
- 4x DC Gear Motors3-Pin Ultrasonic Distance Sensor
- 3x IR Sensors
- Half-size Breadboard
- Jumper Wires
- USB Cable
- Battery / Power Supply

Wiring Diagram:



Schematic Diagram:



Strategy Explanations:

Our robot is designed with both offensive and defensive strategies to ensure stability, control, and effective competition during matches.

For the basic movement strategy, the robot uses a controlled back-and-forth pushing motion instead of a continuous forward push. This approach helps prevent excessive strain on the battery and reduces the risk of power instability during long operation. It also allows better control when engaging with the opponent.

For line detection and boundary control, the robot is equipped with line sensors on both sides. When a black line is detected, the robot automatically moves away from it to avoid going out of bounds or self-elimination. This ensures that the robot always stays within the competition area.

For the defensive strategy, when the robot detects that it is positioned on or near the line, it will rotate its rear side toward the detected area and scan for the opponent. Once the opponent is detected, the robot will face forward and remain stable while tracking the opponent's distance.

As the opponent approaches, the robot continuously computes distance values (e.g., 100, 90, 80, 70, 60). When the opponent reaches a critical range (around 50), the robot will execute a forward attack combined with a quick left or right turn to evade direct collision. After evading, it will realign itself toward the opponent's last known direction and attempt a counterattack.

In another scenario where both robots are pushing head-on, our robot will switch to a controlled reverse mode while maintaining speed and distance relative to the opponent. This ensures stability during a pushing contest. If the rear sensor detects the black line while reversing, the robot will immediately perform a hard left or right turn to avoid exiting the arena.

Problems Encountered:

One of the main issues encountered during development was the risk of battery strain when using continuous forward pushing. This could potentially cause instability or sudden power loss during operation.

Another problem was the possibility of the robot accidentally going out of bounds or self-eliminating if it continuously pushed forward without proper boundary detection.

We also encountered challenges in ensuring stable behavior during close combat situations, especially when both robots are pushing against each other at the front. Without proper control, the robot could lose balance or move out of the arena.

Solutions:

To solve the battery strain issue, we implemented a back-and-forth controlled pushing motion instead of continuous forward movement. This helps distribute power usage more efficiently and prevents overheating or excessive load on the system.

To address boundary risks, we used line sensors on both sides of the robot. When a black line is detected, the robot automatically adjusts its movement away from the boundary, preventing self-elimination.

For improved defensive performance, we programmed the robot to scan, track, and react based on distance sensing. This allows it to dynamically adjust its movement, evade attacks, and counterattack effectively.

During head-on pushing scenarios, the robot uses a reverse-matching strategy, where it maintains relative distance and speed while backing up. If the rear sensor detects the boundary line, the robot immediately performs a hard directional turn to avoid going out of bounds.

Source Code:

```
// --- MOTOR PIN DEFINITIONS (L293D) ---  
  
// Adjust these based on your final wiring  
const int ENB = 5; // Right Motor Speed  
const int IN4 = 6; // Right Motor Backward  
const int IN3 = 7; // Right Motor Forward  
const int IN2 = 8; // Left Motor Backward  
const int IN1 = 9; // Left Motor Forward  
const int ENA = 10; // Left Motor Speed  
  
  
// --- SENSOR PIN DEFINITIONS ---  
  
const int PING_PIN = 11; // 3-Pin Ultrasonic (Signal pin)  
const int IR_LEFT = 2; // Left Line Sensor
```

```
const int IR_RIGHT = 3; // Right Line Sensor
const int IR_REAR = 4; // Rear Line Sensor (Added based on strategy)

// --- CONSTANTS & VARIABLES ---

// Change to LOW or HIGH depending on how your specific IR modules trigger on black
const int BLACK_LINE = LOW;
const int SAFE_SPEED = 150;
const int ATTACK_SPEED = 255;

void setup() {
    Serial.begin(9600);

    // Motor Setup
    pinMode(ENA, OUTPUT);
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);
    pinMode(ENB, OUTPUT);

    // IR Sensor Setup
    pinMode(IR_LEFT, INPUT);
    pinMode(IR_RIGHT, INPUT);
    pinMode(IR_REAR, INPUT);
}

void loop() {
```

```

// 1. Read Sensors

int leftDetect = digitalRead(IR_LEFT);
int rightDetect = digitalRead(IR_RIGHT);
int rearDetect = digitalRead(IR_REAR);
long distance = getDistance();


// -----
// PRIORITY 1: LINE DETECTION & AVOIDANCE (Defensive Strategy)
// -----


if (rearDetect == BLACK_LINE) {
    // Scenario: Reversing during head-on push, rear hits line
    // Action: Immediate hard left/right turn to avoid exiting
    hardTurnLeft(ATTACK_SPEED);
    delay(400); // Tune this delay for a 90-180 degree turn
    return;    // Restart loop
}
else if (leftDetect == BLACK_LINE) {
    // Action: Rotate rear toward the detected area (turn right)
    turnRight(SAFE_SPEED);
    delay(300); // Tune to face rear toward line
    scanForOpponent();
    return;
}
else if (rightDetect == BLACK_LINE) {
    // Action: Rotate rear toward the detected area (turn left)
    turnLeft(SAFE_SPEED);

```

```

    delay(300); // Tune to face rear toward line
    scanForOpponent();
    return;
}

// -----
// PRIORITY 2: OPPONENT ENGAGEMENT (Offensive Strategy)
// -----

if (distance > 0 && distance <= 15) {
    // Scenario: Both robots pushing head-on
    // Action: Controlled reverse mode
    moveBackward(SAFE_SPEED);
}
else if (distance > 15 && distance <= 50) {
    // Scenario: Critical range reached (~50cm)
    // Action: Evasive forward attack (quick left/right, then forward)
    executeEvasiveAttack();
}
else if (distance > 50 && distance <= 100) {
    // Scenario: Tracking distance (100, 90, 80, 70, 60)
    // Action: Controlled back-and-forth pushing motion (Stamina saving)
    moveForward(SAFE_SPEED);
    delay(200);
    moveBackward(SAFE_SPEED / 2); // Slight backup
    delay(100);
}

```

```

else {
    // Scenario: No opponent detected

    // Action: Spin to search
    turnRight(SAFE_SPEED);
}
}

// =====

// CUSTOM FUNCTIONS FOR BEHAVIORS

// =====

// Reads distance from a 3-pin PING))) Ultrasonic Sensor
long getDistance() {
    pinMode(PING_PIN, OUTPUT);
    digitalWrite(PING_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(PING_PIN, HIGH);
    delayMicroseconds(5);
    digitalWrite(PING_PIN, LOW);

    pinMode(PING_PIN, INPUT);
    long duration = pulseIn(PING_PIN, HIGH, 30000); // 30ms timeout

    if (duration == 0) return 999; // No echo
    return duration / 29 / 2;    // Convert to cm
}

```



```
// Evasive counterattack maneuver
void executeEvasiveAttack() {
    // Quick turn to evade direct collision
    turnLeft(ATTACK_SPEED);
    delay(150); // Tune for a quick dodge
```

```
// Forward attack / counterattack
    moveForward(ATTACK_SPEED);
    delay(400); // Surge forward
}
```

```
// Scan until opponent is found
void scanForOpponent() {
    while (getDistance() > 100) {
        turnRight(SAFE_SPEED); // Keep spinning until target acquired
    }
    stopMotors(); // Face forward and remain stable tracking distance
    delay(100);
}
```

```
// --- BASIC MOTOR CONTROLS ---
```

```
void moveForward(int speed) {
    analogWrite(ENA, speed);
    analogWrite(ENB, speed);
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
```

```
digitalWrite(IN3, HIGH);  
digitalWrite(IN4, LOW);  
}
```

```
void moveBackward(int speed) {  
    analogWrite(ENA, speed);  
    analogWrite(ENB, speed);  
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, HIGH);  
    digitalWrite(IN3, LOW);  
    digitalWrite(IN4, HIGH);  
}
```

```
void turnLeft(int speed) {  
    analogWrite(ENA, speed);  
    analogWrite(ENB, speed);  
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, HIGH);  
    digitalWrite(IN3, HIGH);  
    digitalWrite(IN4, LOW);  
}
```

```
void turnRight(int speed) {  
    analogWrite(ENA, speed);  
    analogWrite(ENB, speed);  
    digitalWrite(IN1, HIGH);  
    digitalWrite(IN2, LOW);  
}
```

```
digitalWrite(IN3, LOW);  
digitalWrite(IN4, HIGH);  
}
```

```
void hardTurnLeft(int speed) {  
    // Overdrive one side backwards, one side forwards hard  
    analogWrite(ENA, speed);  
    analogWrite(ENB, speed);  
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, HIGH);  
    digitalWrite(IN3, HIGH);  
    digitalWrite(IN4, LOW);  
}
```

```
void stopMotors() {  
    analogWrite(ENA, 0);  
    analogWrite(ENB, 0);  
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, LOW);  
    digitalWrite(IN3, LOW);  
    digitalWrite(IN4, LOW);  
}
```

Robot Images:

